Cocoa: Towards a Scalable Compute Cost-aware Data Analytics System

1st Kwangsung Oh
Computer Science
University of Nebraska at Omaha
Omaha, US
kwangsungoh@unomaha.edu

2nd Myoungkyu Song
Computer Science
University of Nebraska at Omaha
Omaha, US
myoungkyu@unomaha.edu

Abstract

Recently, many data analytics systems have focused on adopting a newly emerging compute resource, *serverless*, which offers scalability and agility to deal with peak workloads in a *timely* and *cost-efficient* manner, i.e., serverless data analytics (SDA). Unfortunately, these systems may encounter a *cost bottleneck* ($) because they have ignored the per unit time cost ($) of serverless, which is more expensive by up to 5.8 times for the same compute capacity than a traditional compute resource such as a virtual machine (VM). In addition, SDA may also encounter a *performance bottleneck* due to serverless' worse performance than VM. In this paper, we first study and report when serverless is beneficial for data analytics. Then, we present a scalable compute cost-aware data analytics system, *Cocoa*, that exploits serverless and VM together to achieve composite benefits. A Cocoa prototype was implemented on Spark. Evaluation results show a richer cost-performance tradeoff space opened by exploiting heterogeneous compute resources together, and identify substantial opportunities for future serverless-enabled systems research.

I. Introduction

Serverless compute resource (serverless) such as Apache OpenWhisk [3], AWS Lambda [4], Azure Functions [7], and Google Functions [13] have gained attention from Internet applications thanks to their advantages such as scalability, agility, easy maintenance, and cost ($) efficiency[1]. Serverless allows software developers to focus on implementing function logic for their applications to handle events, e.g., requests from users, and cloud providers to handle resource provisioning, scaling, and managing. Thus, varying (peak) workloads can be handled automatically and quickly without additional effort. Because serverless charges only when the function code is executed at 100 millisecond granularity, i.e., pure pay-as-you-go [4], [7], [13], applications can deal with diverse workloads in a *timely* and *cost-efficient* manner as shown in many recent works [12], [14], [18], [45], [48].

Serverless is also appealing for data analytics, one of the most important workloads in a cloud environment, because its workloads fit well for serverless, i.e., numerous independent (stateless) tasks that execute in parallel. While data analytics systems usually require *overprovisioned* compute resources such as a virtual machine (VM) to handle *peak workload* without a performance bottleneck, serverless can allow data analytics systems to avoid deploying redundant compute resources a priori and thus achieve cost efficiency. However, serverless has several constraints, e.g., small memory size, limited network communication, and volatile and small storage, that must be overcome for data analytics to realize the benefits of serverless.

TABLE I
Compute resource cost ($) per second for a single CPU core (vCPU) and 1 GB memory. Note: serverless cost is adjusted to "per second billing" for comparison.

Region \ Compute	AWS		Azure	
	Serverless	VM	Serverless	VM
US East	$1.67e-5	**$2.88e-6**	$1.6e-5	**$2.88e-6**
SA East	$1.67e-5	**$4.66e-6**	$1.6e-5	**$4.66e-6**
AP NE	$1.67e-5	**$3.77e-6**	$1.6e-5	**$3.77e-6**

To address this problem, many serverless data analytics (SDA) systems [19], [21], [23], [35], [37], [38] have been introduced. These systems have focused on supporting persistent storage and network communication between serverless instances by utilizing external storage systems, e.g., AWS S3 [5], CouchDB [10], Redis [36], and cloud queuing services (e.g., AWS SQS [6]). While these SDA systems allow applications to handle peak workloads such as ad-hoc queries quickly and cost-efficiently without overprovisioning VM, they may encounter a *cost bottleneck* because they have ignored the expensive per unit time cost ($) of serverless.

One may think that using serverless results in reduced cost compared to VM, as serverless charges only when the function code is executed while VM charges while the instances are allocated even when idle. Yet, serverless has a more expensive unit time cost **3.4X** $\sim$ **5.8X** than VM based on regions as shown in Table I[2]. Thus, SDA systems may encounter a cost bottleneck if they simply utilize serverless to handle queries without consideration of heterogeneous compute costs, as discussed in previous work [24].

[1]The term cost is used to refer to the monetary cost of cloud resources unless otherwise mentioned.

[2]We use AWS t3.small (2 cores with 2 GB) and Azure B1S (1 core with 1 GB) for VM cost.

In terms of performance, serverless may provide worse (or similar, at best) performance than VM because serverless instances run on VM instances [26], [46]. It is known that serverless CPU performance is determined by memory size (and thus cost), e.g., AWS Lambda provides a single full vCPU core with memory of at least 1739 MB [9]. This implies that applications need to pay more cost than the costs shown in Table I to acquire better performance from serverless. Our experimental results in Section II confirm that serverless provides 25 $\sim$ 30% worse performance than VM for CPU-intensive workloads with the same amount of compute resources: a single vCPU core and 1 GB memory.

In this paper, we argue that data analytics systems must consider *heterogeneous compute resources characteristics*, e.g., cost, performance, scalability, and agility, to avoid any cost/performance bottlenecks while meeting applications' desired goals. To consider heterogeneous compute resources, we are motivated by the following questions:

- Which compute resources are suitable for peak workloads?
- What is the minimum cost ($) to meet target query performance (or vice versa)?
- How can data analytics explore the tradeoff space opened by exploiting heterogeneous compute resources?

To answer these questions, we have designed and implemented `Cocoa`, a scalable **CO**mpute **CO**st-**A**ware data analytics system. The goal of Cocoa is to handle peak workloads with minimized cost while meeting applications' target performance goals by exploiting both serverless and VM together.

A prototype implementation of Cocoa was built on the Spark [50] framework. We evaluated Cocoa on AWS using several queries of the well-known benchmark TPC-DS [42]. Experimental results show that using heterogeneous compute resources opens a richer cost-performance tradeoff space, and Cocoa allows applications to easily explore this space. To the best of our knowledge, Cocoa is the first system that explores a cost-performance tradeoff space opened by exploiting serverless and VM together. We do not propose a new algorithm for determining optimal compute resource configurations such as the number of VM instances and types, as done in previous works [2], [8], [22], [27], [33], [43], [47]. Instead, we focus on showing the potential opportunities from exploiting heterogeneous compute resources together to achieve diverse cost-performance goals.

The main contributions of this paper are:

- The observation that cost efficiency of serverless does not necessarily reduce the overall cost due to heterogeneous compute resources' performance and cost.
- The design and implementation of Cocoa, the data analytics system that explores cost-performance tradeoff space opened by exploiting serverless and VM together.
- The experimental results in a real cloud environment, which identify substantial opportunities for future serverless-enabled systems research.

II. Serverless vs. Virtual Machine

In this section, we compare serverless and virtual machine (VM) in terms of scalability, agility, cost, and performance to show the benefits from exploiting heterogeneous compute resources for data analytics systems.

Scalability and agility: Data analytics usually requires *over-provisioned* compute resources such as redundant VM instances to handle peak workloads while avoiding performance degradation. This approach is simple and works well, but additional costs for unused resources are incurred. To handle peak workloads in a cost-efficient way, many previous works [14], [15], [20], [49] have tried to scale compute resources based on given workloads, i.e., deploying additional VM instances. These systems, however, may not handle workload changes quickly because they use VM instances that incur significant overhead, i.e., cold-boot latency, that may take up to 95 seconds [28]. Note that we observe that AWS offers reduced VM cold-boot latency ($<$ 60 seconds) in our experiments, while Azure offers higher latency ($>$ 120 seconds). In either case, there is still significant overhead to handle peak workloads quickly. Due to the overhead, these systems may not achieve an application's desired performance goals. On the other hand, a new emerging compute resource, serverless, can start running function code quickly ($\sim$10 ms), which allows data analytics to start executing tasks quickly in parallel with monopolized compute resources for each task. Thus, *serverless is appealing for data analytics to handle peak workloads in a timely manner*.

Cost ($): Most cloud service providers charge for VM each time a new instance is launched at one second granularity for a minimum of 60 seconds even if the instance remains idle, i.e., cost inefficiency. On the other hand, serverless instances get charged only when function code is invoked at 100 millisecond granularity based on memory size, i.e., cost efficiency. However, the per unit time cost of serverless is more expensive (**3.4X** $\sim$ **5.8X**) compared to VM for the same compute resource capacity as shown in Table I. Thus, applications that have a tight budget may encounter a *cost bottleneck* if they use serverless to handle peak workloads without consideration of heterogeneous pricing policies. Both serverless and VM instances require additional storage costs, e.g., Redis or AWS S3, for serverless and local disk for VM, and thus we consider these costs in this work. In short, *VM offers a much cheaper unit time cost than serverless, while serverless offers better cost efficiency than VM*.

Performance: Figure 1 shows serverless performance with varying memory sizes of AWS Lambda [4] using a Fibonacci function (Fibonacci) (100 times run with 30 as a parameter) written in Python 3.6 and a popular benchmark tool, Sysbench [40]. The figure clearly shows that serverless performance is improved as memory size increases, i.e., reduced execution time for Fibonacci and increased throughput for Sysbench. Note, we can observe that each serverless instance provides dual vCPU cores. The figure shows that specific memory size (more than 1.5 GB) is required to achieve a full single core

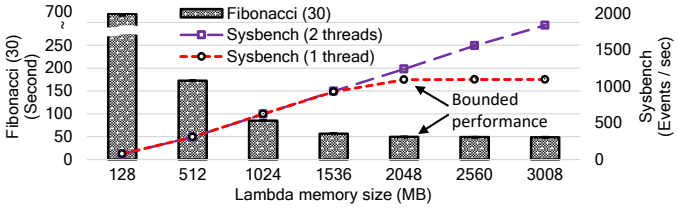
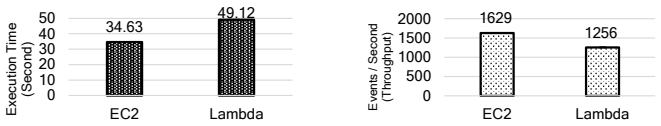


Fig. 1. AWS Labmda performance comparisons based on memory size. Cost ($) is proportional to memory size.



(a) Fibonachi (Lower is better) (b) Sysbench (Higher is better)

Fig. 2. Performance comparison between VM (AWS EC2:t3.small) and serverless (AWS Lambda:2 GB) with dual vCPU and 2 GB memory.

performance as shown in [9]. We also make an interesting observation that a single-threaded task cannot fully utilize the CPU performance of a serverless instance with more than 2 GB of memory. That is, Figure 1 shows that single-threaded task performance is bounded to memory size (2 GB), i.e., Fibonacci and Sysbench with a single thread option, while performance of a multi-threaded task is improved proportionally to memory size, i.e., Sysbench with a 2 threads option. This shows that multi-threaded tasks can fully utilize performance of a serverless instance with more than 2 GB memory.

Since serverless instances are running on VM instances [26], [46], serverless may naturally provide worse performance than VM due to the overhead of additional abstraction. Figure 2 shows performance comparisons between serverless and VM systems with the same compute resource capacity (dual vCPU cores and 2GB memory) using Fibonacci and Sysbench (with a 2 threads option). The results clearly show that VM provides better performance than serverless for the same amount of compute resource. While a serverless instance with 3 GB memory can provide better performance than VM for multiple-threaded tasks (Sysbench with a 2 threads option) as shown in Figure 1, this requires 50% additional cost, which makes serverless even more expensive by **5.4X** $\sim$ **8.7X** than VM. Note, we can see a similar pattern of results from other serverless platforms such as Apache OpenWhisk [3] and Azure Function [7]. In short, *VM offers better performance than serverless systems with less cost*.

III. When is Using Serverless Beneficial for Data Analytics?

In this section, we illustrate when using serverless provides benefits for data analytics based on what we explained in Section II. Let us assume that an ad-hoc query that causes peak workload is sent to a data analytics system (DAS) as an example. A performance predictor [2], [8], [22], [27], [33], [43], [47] in DAS determines the optimal number of VM instances (N) and predicts the best query execution time (T). Based

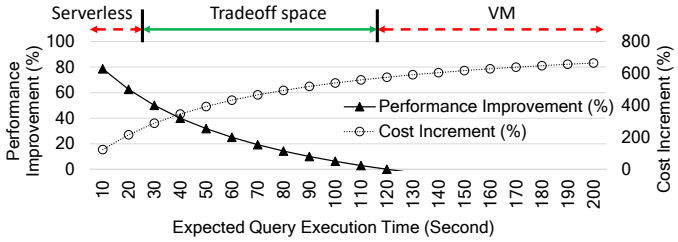


Fig. 3. Performance improvement and cost inflation when using the serverless-only approach with varying query execution time over the VM only approach.

on the decision, DAS can spawn N either VM or serverless instances to handle the query without overprovisioned compute resources, i.e., a VM-only approach [14], [15], [20], [49] or a serverless-only approach [19], [21], [35], [37], [38].

Figure 3 shows the performance improvement (left y-axis) and cost increase (right y-axis) when DAS uses the serverless-only approach over the VM-only approach with varying predicted execution time T (x-axis). Note, cold-boot time (60 seconds) is added to the VM-only case and 30% additional execution time is added to the serverless-only case due to heterogeneous boot-up latency and performance as explained in Section II. For cost comparison, we use compute resource costs of a US-East region shown in Table I. The figure shows that if the query is expected to have a short query execution time, e.g., T $<$ 10 seconds, using the serverless-only approach improves overall performance significantly (80%) with little additional cost (10%) compared to the VM-only approach. However, the serverless-only approach results in poorer performance and significantly inflates overall costs for long-running queries due to its worse performance and more expensive cost than VM. That is, the serverless-only approach results in 6X cost for worse performance compared to the VM-only approach if T $\geq$ 120 seconds. These results clearly show that *serverless is suitable for short-running queries thanks to its agility, and VM is suitable for long-running queries* as shown at the top of Figure 3. Lastly, the figure shows that there is *a richer cost-performance trade-off space* between two approaches, which most applications likely want to explore while meeting their target performance or cost goals by exploiting VM and serverless together.

Limitations of serverless: While serverless has several limitations such as limited network communication, cold-boot time, and limited execution time, we ignore them as addressed in previous works [1], [23], [30], [31], [35], [39], [46].

IV. Exploring Tradeoff Space Problem Model

In this work, our goal is to allow applications that generate MapReduce-like ad-hoc queries, to explore the richer cost-performance tradeoff space shown in Figure 3. To handle incoming queries without overprovisioned compute resources, Cocoa spawns N serverless instances and M VM instances to meet applications' goals. We will show how N and M are determined based on inputs in the following section.

TABLE II
INPUTS FOR COCOA

Notation	Description
N_{max}	The maximum number of instances for a query
N_{task}	Total number of tasks
T_{vm}^{task}	Time for a task with a single core of VM
T_{sl}^{task}	Time for a task with a single core of serverless = $T_{vm}^{task} \cdot SL_{overhead}$
$SL_{overhead}$	Performance overhead of serverless over VM
T_{vm_cold}	Cold boot time for VM
$C_{vm}^{compute}$	Compute cost for a single VM instance per second
$C_{vm}^{storage}$	Storage cost for a single VM instance per second
$C_{sl}^{compute}$	Compute cost for a single serverless instance per second
$C_{external}$	External storage (Redis or AWS S3) cost
$Threshold$	Tolerable additional latency threshold

A. Inputs and output

Table II shows the inputs that Cocoa uses. In this work, we assume that some performance-related inputs N_{max}, N_{task}, and T_{vm}^{task} will be provided by other performance prediction systems [2], [8], [22], [27], [33], [43], [47]. For N_{max}, we assume that each instance has a single vCPU core and 1 GB memory. For simplicity, we also assume that N_{task} is the total number of tasks for a query including the map and shuffle stages, and T_{vm}^{task} is the average execution time for a task. While $SL_{overhead}$ and T_{vm_cold} can be monitored and updated continuously, we set $SL\_overhead$ to 1.3 (30% performance overhead) and T_{vm_cold} to 60 seconds (cold boot latency) by default as shown in Section II. Applications are required to provide their performance goals by setting $Threshold$. The value of $Threshold$ determines a target query latency that is used as a constraint in our problem model. While Cocoa requires compute resource cost information, the costs are rarely changed and are thus static in this work.

Given these inputs, Cocoa computes the number of serverless instances (N_{sl}) and VM instances (N_{vm}). Given this output, Cocoa sends requests to cloud providers to create new serverless and VM instances dynamically to handle the query.

B. Optimization Problem Formulation

We solve two constrained optimization problems to make the compute resource decision pair (N_{sl}, N_{vm}) using mixed integer programming (MIP).

1) Determining minimized query latency: The first problem is to find the minimum latency of a given query.

Variable: We define a single output variable:

$$\forall i < N_{max}, \forall j < N_{max} : N_{ij}$$

N_{ij} are binary variables (0 or 1): if 1, i serverless instances and j VM instances are created. Given the variable, the numbers of serverless instances (N_{sl}) and VM instances (N_{vm}) are $\sum_i \sum_j N_{ij} \cdot i$ and $\sum_i \sum_j N_{ij} \cdot j$, respectively.

Constraints:

$$\forall i < N_{max}, \forall j < N_{max} : \sum N_{ij} = 1$$

This is a constraint where only one set of decision variables can be determined.

$$N_{sl} + N_{vm} \geq 1$$

$$N_{sl} + N_{vm} \leq N_{task}$$

$$N_{sl} + N_{vm} \leq N_{max}$$

These are constraints such that there should be at least one instance to make progress, and the total number of instances should be less than N_{task} and N_{max} to avoid a cost bottleneck.

In our model, we divide the query execution into two stages, the *pre-stage* and the *post-stage*, since VM instances cannot execute tasks during T_{vm_cold}. In the pre-stage, only serverless instances can execute tasks due to its agility while VM instances are being created. In the post-stage, both serverless and VM instances can execute tasks.

Objective: Minimize total query latency (T_{total}^{exec}) = pre-stage time (T_{pre}^{exec}) + post-stage time (T_{post}^{exec}).

To get execution time for each stage, we estimate how many tasks can be done in the pre-stage given N_{sl}. The number of tasks in the pre-stage (N_{pre}^{task}) can be determined as follows.

$$N_{pre}^{task} = N_{sl} \cdot T_{vm\_cold} / T_{sl}^{task}$$

Note, N_{pre}^{task} will be 0 if no serverless instances are used for a query, i.e., $N_{sl} = 0$. If N_{pre}^{task} is greater than the total tasks (N_{task}), N_{pre}^{task} is set to N_{task} for the formulation, i.e., the query is done in the pre-stage. Given N_{pre}^{task}, we can estimate T_{pre}^{exec} as follows.

$$T_{pre}^{exec} = \begin{cases} T_{vm\_cold} & \text{if } N_{sl} = 0, \\ N_{task}/(N_{sl}/T_{sl}^{task}) & \text{if} N_{pre}^{task} \geq N_{task}, \\ N_{pre}^{task}/(N_{sl}/T_{sl}^{task}) & \text{otherwise} \end{cases}$$

The number of tasks for the post-stage (N_{post}^{task}) can be easily calculated, i.e., $N_{post}^{task} = N_{task} - N_{pre}^{task}$. Since both serverless and VM instances can execute tasks in the post-stage, we calculate the average execution time for a single task in the post-stage, i.e., $T_{post}^{task} = (N_{sl}/T_{sl}^{task} + N_{vm}/T_{vm}^{task})$. Given N_{post}^{task} and T_{post}^{task}, we can estimate T_{post}^{exec} as follows.

$$T_{post}^{exec} = \begin{cases} 0 & \text{if } N_{post}^{task} = 0, \\ N_{post}^{task}/T_{post}^{task} & \text{otherwise} \end{cases}$$

By solving this problem, we can get a pair (N_{sl}, N_{vm}) with which we can estimate the minimized query latency (Min_{lat}).

2) Minimizing cost for a target query latency: The second problem is to determine a pair (N_{sl}, N_{vm}), that meets applications' desired cost-performance trade-off preferences ($Threshold$). While we use the same variable and constraints used in the first problem, we add a performance constraint and use a different objective. Given Min_{lat} from the first problem and $Threshold$ from applications, the target latency ($Target_{lat}$) is computed as follows.

$$Target_{lat} = Threshold \cdot Min_{lat}$$

Additional constraint:

$$T_{total}^{exec} \leq Target_{lat}$$

This is a constraint to set the target performance goal.

Objective: Minimize Total cost = Compute cost + Storage cost, where

$$Compute_{Cost} = T_{total}^{exec} \cdot (N_{sl} \cdot C_{sl}^{compute} + N_{vm} \cdot C_{vm}^{compute})$$

$$Storage_{Cost} = T_{total}^{exec} \cdot (\delta \cdot N_{sl} \cdot C_{external} + N_{vm} \cdot C_{vm}^{storage})$$

where δ indicates whether there is at least one serverless instance for a query. By solving this problem with the performance constraint, we can get a pair (N_{sl}, N_{vm}) that minimizes cost for a query while meeting a desired target query latency.

V. Experimental Results

A. Experimental Setting

We have implemented the Cocoa prototype on top of Apache Spark (2.2.1) [50]. For a new query, Cocoa models a cost-performance trade-off problem as a mixed integer programming (MIP) problem using PuLP [29], and uses CPLEX [11] to solve the problem as explained in Section IV. We deployed and evaluated Cocoa on the AWS data center in the US East-Virginia (us-east-1) region. We used t3.xlarge (4 vCPU cores and 16 GB of RAM) as a centralized node for Spark master, Spark driver, and Redis [36]. For Spark workers dynamically created, we used t3.small (2 cores and 2 GB memory) for VM and AWS Lambda with 2 GB memory for serverless. Since Lambda offers 2 cores per each invocation, $N_{sl}/2$ serverless and $N_{vm}/2$ VM instances are spawned in our evaluation. We enabled serverless instances to access VM instances by deploying them in the same virtual private cloud.

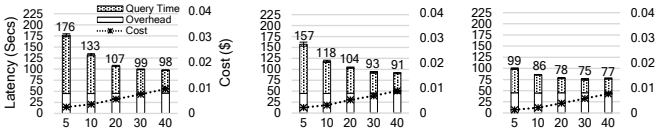
For workload, we used TPC-DS [42], a standard decision support benchmark, at scale factor 100 (100 GB). We used AWS S3 [5] for input TPC-DS datasets, and Redis for sharing intermediate data only when at least one executor is running on a serverless instance. That is, the VM-only approach does not require external storage. While we show the results of 3 TPC-DS queries, 11, 49, and 82, due to space constraints, we could see similar patterns of results from other queries.

We used serverless-only and VM-only approaches as baselines. We varied the $Threshold$ value from 1 (best performance) to 2 (degraded performance for cost reduction). We ran each query 6 times, discarded the worst performance run, took the averages of the rest, and plotted with 95% confidence intervals. The performance results include overhead such as the HiveQL (Spark SQL) [41] compilation, jobs (stages) scheduling, and the cold-boot latency. Though MIP is not efficient, the time for solving the problem has a negligible impact on the overall performance, as Cocoa can solve the optimization problem in less than 0.5 seconds with t3.xlarge.

B. Varying Numbers of VM and Serverless Instances

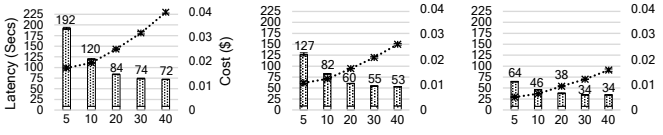
In this section, we show the potential cost-performance tradeoff space for processing queries. Figures 4 and 5 show the query latencies and overall costs using VM-only and serverless-only approaches with varying numbers of instances. Note that using a very few number of instances results in significant performance degradation without cost reduction due to inflated execution time caused by a lack of compute resources. For example, we found that using a single serverless instance for query 82 results in similar cost but 3X execution time compared to using 5 serverless instances. Thus, we set our minimum number of instances to 5 in our evaluation.

For performance, results clearly show that query execution time is reduced as the number of instances is increased. For



(a) Query 11 (b) Query 49 (c) Query 82

Fig. 4. Query latencies with varying number of VM instances.



(a) Query 11 (b) Query 49 (c) Query 82

Fig. 5. Query latencies with varying number of serverless instances.

cost, results show that cost is increased as the number of instances is increased, i.e., performance improvement with additional cost. The results also show the limitation of speedup by increasing the number of instances. That is, using 40 instances does not yield better performance than using 30 instances even with more cost. Thus, we conclude that 60 cores (30 instances) are enough to fully parallelize queries in our experimental environment, and we set the N_{max} value to 60 for the rest of experiments. Note that we could see that Redis is not a performance bottleneck from all the cases (95% CPU usage of Redis at peak).

Table III shows the performance improvement and cost increase when we use the serverless-only approach compared to the VM-only approach based on results shown in Figures 4 and 5. The results show that the serverless-only approach (Figure 5) offers performance improvement (up to 55.5%) with additional cost (up to 535%) in most cases thanks to agility and expensive cost, which shows the substantial cost-performance tradeoff space. For example, applications may want to achieve 30% performance improvement with minimized additional cost for query 49 by exploiting serverless and VM together. Interestingly, we observe that the serverless-only approach can offer worse performance (-8.8%) even with significantly inflated cost (684%), i.e., query 11 with 5 instances case. This is because serverless instances are used for a longer time than other cases, showing that *serverless is not suitable for long-running queries* as we discussed in Section III. The table also shows varying performance improvement and cost increases based on queries. With 30 instances, for example, applications can achieve 54% performance improvement with 213% additional cost for query 82 while 420% more cost is required to achieve 25.7% performance improvement for query 11. This shows that different queries open up a diverse tradeoff landscape and thus a data analytics system should consider query workload in order to meet desired tradeoff preferences.

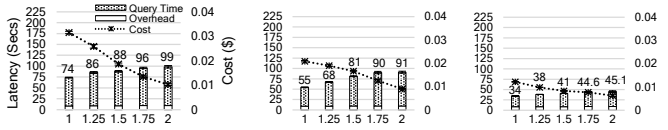
Overall, these experimental results show that *exploiting heterogeneous compute resources and varying their capacities can open a richer cost-performance tradeoff space*.

C. Exploring Tradeoff Space

In this experiment, we compare performance and costs with varying $Threshold$ (1 $\sim$ 2) explained in Section IV with

TABLE III
PERFORMANCE IMPROVEMENT (PI) AND COST INCREASE (CI) OF THE SERVERLESS-ONLY APPROACH COMPARED TO THE VM-ONLY APPROACH WITH VARYING NUMBER OF INSTANCES.

Query \ Inst. #	5		10		20		30		40	
	PI	CI	PI	CI	PI	CI	PI	CI	PI	CI
11	**-8.8%**	684%	9.7%	535%	21.9%	441%	25.7%	420%	26.2%	415%
49	19.4%	494%	30.5%	398%	41.8%	312%	41.3%	305%	41.9%	297%
82	35%	396%	46%	300%	51.1%	243%	54%	213%	55.5%	207%



(a) Query 11 (b) Query 49 (c) Query 82

Fig. 6. Performance and cost comparisons with varying the target latency goal ($Threshold$), i.e., exploring a cost-performance tradeoff space.

baselines, i.e., serverless-only and VM-only approaches. We use the same setting and environment used in Section V-B, but we set the N_{max} value to 60 (30 instances) as mentioned in the previous section. Note, the $Threshold$ = 1 case can be considered the serverless-only approach, as we found that only serverless instances are used in this case, and the $Threshold$ = 2 case can be considered to be close to the VM-only approach as VM instances are mainly used in this case.

Figure 6 shows the query execution times and costs for queries with varying the $Threshold$ values. Not surprisingly, $Threshold$ = 1 case provides the lowest latency (best performance) with highest cost. This is because only serverless instances are chosen and used to maximize performance for all queries without consideration of cost. The results show that the query latencies increase for cost reduction as the $Threshold$ value increases. For the fast-running query, i.e., 82, we found that Cocoa mainly reduces the number of serverless instances to reduce overall cost without using VM due to the significant cold-boot latency. For other queries, we observe that Cocoa utilizes more VM instances as the $Threshold$ value increases to reduce cost while achieving the performance goal.

Overall, these experimental results show that *Cocoa can allow applications to easily explore the richer cost-performance tradeoff options by exploiting serverless and VM together and varying compute resource capacities*.

VI. LIMITATIONS AND RESEARCH CHALLENGES

While we have focused on exploiting heterogeneous compute resources for data analytics, many potential research problems remain.

Workload prediction: We assume that accurate inputs such as the optimal number of compute instances and execution time are available from previous works [2], [8], [22], [27], [33], [43], [47], which may not be true. We plan to implement a workload predictor that considers heterogeneous compute resources to determine optimal compute resource configurations.

Sharing compute resources for multi queries: In this work, Cocoa spawns compute resources for each query and terminates when the query is complete by default. However, significant cost reduction opportunities may be missed as resources are not shared by different queries. For example, two queries may be executed concurrently by sharing compute resources without much contention if they have different workloads, i.e., a compute-intensive workload or a bandwidth-intensive workload. Minimizing cost while satisfying performance goals by sharing both long- and short-lived compute resources for multiple queries would be an interesting research problem.

Single point of performance bottleneck: Due to the serverless design, using external storage is unavoidable for data analytics systems to utilize serverless. While we used Redis to avoid performance bottleneck, Redis would easily become a single point of performance bottleneck if data size becomes larger and more compute resources are used. Achieving efficient and effective communication between serverless instances is desirable for serverless-enabled systems.

Dynamics: We could observe performance variation during our system evaluation, e.g., varying boot-up times for VM instances and query latencies over time. If any significant dynamics are not handled properly, data analytics systems may encounter cost- and performance bottlenecks. Achieving target cost/performance goals, even in the presence of dynamics, would be an important task.

VII. RELATED WORK

Heterogeneous compute resources for data analytics: Recent serverless-enabled data analytics systems [19], [21], [23], [35], [37], [38] may encounter cost- and performance bottlenecks due to heterogeneous performance and the cost of serverless. While recent works [14], [18] have exploited VM and serverless together, they ignore poorer performance of serverless and the cost-performance tradeoff space.

Optimal cloud configurations for data analytics: Many previous works [2], [8], [22], [27], [33], [43], [47] have focused on determining optimal cloud configurations. These works, however, have only considered VM, and thus cannot get the benefits of serverless. We plan to implement a workload predictor to determine the optimal numbers of serverless and VM instances to meet applications' desired goals.

Geo-distributed data analytics: Many geo-distributed data analytics (GDA) systems [16], [25], [32], [34], [44] have been introduced to improve the performance for processing geo-distributed data while overcoming the limitation of a wide area network. One recent GDA system, Tetrium [17], considered heterogeneous compute capacities at different regions as well. We plan to extend Cocoa for GDA systems to exploit serverless, which allows applications to explore the richer cost-performance tradeoff space in a GDA environment.

VIII. CONCLUSION

In this paper, we study and report on heterogeneous compute resources characteristics, which show a richer cost-performance trade-off space opened by exploiting them together, and present Cocoa, which explores the trade-off space. The experimental results show that Cocoa enables applications to easily explore the cost-performance tradeoff space, which identifies substantial opportunities for future serverless-enabled systems research.

References

[1] I. E. Akkus, R. Chen, I. Rimac, M. Stein, K. Satzke, A. Beck, P. Aditya, and V. Hilt. SAND: Towards high-performance serverless computing. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 923–935, Boston, MA, July 2018. USENIX Association.

[2] O. Alipourfard, H. H. Liu, J. Chen, S. Venkataraman, M. Yu, and M. Zhang. Cherrypick: Adaptively unearthing the best cloud configurations for big data analytics. In *Proceedings of the 14th USENIX Conference on Networked Systems Design and Implementation*, NSDI'17, pages 469–482, Berkeley, CA, USA, 2017. USENIX Association.

[3] Apache OpenWhisk. http://https://openwhisk.apache.org/.

[4] AWS Lambda Pricing. https://aws.amazon.com/lambda/.

[5] AWS S3. https://aws.amazon.com/s3/.

[6] AWS Simple Queue Service. https://aws.amazon.com/sqs/.

[7] Azure Functions. https://azure.microsoft.com/en-us/services/functions/.

[8] M. Bilal, M. Canini, and R. Rodrigues. Finding the right cloud configuration for analytics clusters. In *Proceedings of the 11th ACM Symposium on Cloud Computing*, SoCC '20, page 208–222, New York, NY, USA, 2020. Association for Computing Machinery.

[9] Configuring functions in the AWS Lambda console. https://docs.aws.amazon.com/lambda/latest/dg/configuration-console.html/.

[10] CouchDB. https://couchdb.apache.org/.

[11] CPLEX Optimizer. https://goo.gl/6BKOZY.

[12] S. Fouladi, R. S. Wahby, B. Shacklett, K. V. Balasubramaniam, W. Zeng, R. Bhalerao, A. Sivaraman, G. Porter, and K. Winstein. Encoding, fast and slow: Low-latency video processing using thousands of tiny threads. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 363–376, Boston, MA, Mar. 2017. USENIX Association.

[13] Google Cloud Functions. https://https://cloud.google.com/functions/.

[14] J. R. Gunasekaran, P. Thinakaran, M. T. Kandemir, B. Urgaonkar, G. Kesidis, and C. Das. Spock: Exploiting serverless functions for slo and cost aware resource procurement in public cloud. In *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*, pages 199–208, 2019.

[15] A. Harlap, A. Chung, A. Tumanov, G. R. Ganger, and P. B. Gibbons. Tributary: spot-dancing for elastic services with latency slos. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 1–14, Boston, MA, July 2018. USENIX Association.

[16] B. Heintz, A. Chandra, R. K. Sitaraman, and J. Weissman. End-to-end optimization for geo-distributed mapreduce. *IEEE Transactions on Cloud Computing*, 4(3):293–306, 2016.

[17] C.-C. Hung, G. Ananthanarayanan, L. Golubchik, M. Yu, and M. Zhang. Wide-area analytics with multiple resources. In *Proceedings of the Thirteenth EuroSys Conference*, EuroSys '18, pages 12:1–12:16, New York, NY, USA, 2018. ACM.

[18] A. Jain, A. F. Baarzi, G. Kesidis, B. Urgaonkar, N. Alfares, and M. Kandemir. Splitserve: Efficiently splitting apache spark jobs across faas and iaas. In *Proceedings of the 21st International Middleware Conference*, Middleware '20, page 236–250, New York, NY, USA, 2020. Association for Computing Machinery.

[19] E. Jonas, Q. Pu, S. Venkataraman, I. Stoica, and B. Recht. Occupy the cloud: Distributed computing for the 99%. In *Proceedings of the 2017 Symposium on Cloud Computing*, page 445–451, New York, NY, USA, 2017. Association for Computing Machinery.

[20] A. Khandelwal, R. Agarwal, and I. Stoica. Blowfish: Dynamic storage-performance tradeoff in data stores. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*, pages 485–500, Santa Clara, CA, Mar. 2016. USENIX Association.

[21] Y. Kim and J. Lin. Serverless data analytics with flint. In *2018 IEEE 11th International Conference on Cloud Computing (CLOUD)*, pages 451–455, 2018.

[22] A. Klimovic, H. Litz, and C. Kozyrakis. Selecta: Heterogeneous cloud storage configuration for data analytics. In *Proceedings of the 2018 USENIX Conference on Usenix Annual Technical Conference*, USENIX ATC '18, pages 759–773, Berkeley, CA, USA, 2018. USENIX Association.

[23] A. Klimovic, Y. Wang, P. Stuedi, A. Trivedi, J. Pfefferle, and C. Kozyrakis. Pocket: Elastic ephemeral storage for serverless analytics. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, OSDI'18, pages 427–444, Berkeley, CA, USA, 2018. USENIX Association.

[24] J. Kuhlenkamp, S. Werner, and S. Tai. The ifs and buts of less is more: A serverless computing reality check. In *2020 IEEE International Conference on Cloud Engineering (IC2E)*, pages 154–161, 2020.

[25] S. Liu, H. Wang, and B. Li. Optimizing shuffle in wide-area data analytics. In *37th IEEE International Conference on Distributed Computing Systems, ICDCS 2017, Atlanta, GA, USA, June 5-8, 2017*, pages 560–571, 2017.

[26] W. Lloyd, S. Ramesh, S. Chinthalapati, L. Ly, and S. Pallickara. Serverless computing: An investigation of factors influencing microservice performance. In *2018 IEEE International Conference on Cloud Engineering (IC2E)*, pages 159–169, 2018.

[27] A. Mahgoub, A. M. Medoff, R. Kumar, S. Mitra, A. Klimovic, S. Chaterji, and S. Bagchi. OPTIMUSCLOUD: Heterogeneous configuration optimization for distributed databases in the cloud. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 189–203. USENIX Association, July 2020.

[28] M. Mao and M. Humphrey. A performance study on the vm startup time in the cloud. In *2012 IEEE Fifth International Conference on Cloud Computing*, pages 423–430, 2012.

[29] S. Mitchell, M. O'Sullivan, and I. Dunning. Pulp: A linear programming toolkit for python, 2011.

[30] A. Mohan, H. Sane, K. Doshi, S. Edupuganti, N. Nayak, and V. Sukhomlinov. Agile cold starts for scalable serverless. In *11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 19)*, Renton, WA, July 2019. USENIX Association.

[31] E. Oakes, L. Yang, D. Zhou, K. Houck, T. Harter, A. Arpaci-Dusseau, and R. Arpaci-Dusseau. SOCK: Rapid task provisioning with serverless-optimized containers. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 57–70, Boston, MA, July 2018. USENIX Association.

[32] K. Oh, A. Chandra, and J. Weissman. A network cost-aware geo-distributed data analytics system. In *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*, pages 649–658, 2020.

[33] Y. Peng, Y. Bao, Y. Chen, C. Wu, and C. Guo. Optimus: An efficient dynamic resource scheduler for deep learning clusters. In *Proceedings of the Thirteenth EuroSys Conference*, EuroSys '18, New York, NY, USA, 2018. Association for Computing Machinery.

[34] Q. Pu, G. Ananthanarayanan, P. Bodik, S. Kandula, A. Akella, P. Bahl, and I. Stoica. Low latency geo-distributed data analytics. *SIGCOMM Comput. Commun. Rev.*, 45(4):421–434, Aug. 2015.

[35] Q. Pu, S. Venkataraman, and I. Stoica. Shuffling, fast and slow: Scalable analytics on serverless infrastructure. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 193–206, Boston, MA, Feb. 2019. USENIX Association.

[36] Redis. https://redis.io/.

[37] V. Shankar, K. Krauth, Q. Pu, E. Jonas, S. Venkataraman, I. Stoica, B. Recht, and J. Ragan-Kelley. numpywren: serverless linear algebra, 2018.

[38] Spark on Lambda. https://github.com/qubole/spark-on-lambda/.

[39] V. Sreekanti, C. Wu, X. C. Lin, J. Schleier-Smith, J. M. Faleiro, J. E. Gonzalez, J. M. Hellerstein, and A. Tumanov. Cloudburst: Stateful functions-as-a-service, 2020.

[40] sysbench. https://github.com/akopytov/sysbench/.

[41] A. Thusoo, J. S. Sarma, N. Jain, Z. Shao, P. Chakka, N. Zhang, S. Antony, H. Liu, and R. Murthy. Hive - a petabyte scale data warehouse using hadoop. In *2010 IEEE 26th International Conference on Data Engineering (ICDE 2010)*, pages 996–1005, 2010.

[42] TPC-DS. http://www.tpc.org/tpcds/.

[43] S. Venkataraman, Z. Yang, M. Franklin, B. Recht, and I. Stoica. Ernest: Efficient performance prediction for large-scale advanced analytics. In *Proceedings of the 13th Usenix Conference on Networked Systems Design and Implementation*, NSDI'16, pages 363–378, Berkeley, CA, USA, 2016. USENIX Association.

[44] R. Viswanathan, G. Ananthanarayanan, and A. Akella. CLARINET: wan-aware optimization for analytics queries. In *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016.*, pages 435–450, 2016.

[45] A. Wang, J. Zhang, X. Ma, A. Anwar, L. Rupprecht, D. Skourtis, V. Tarasov, F. Yan, and Y. Cheng. Infinicache: Exploiting ephemeral serverless functions to build a cost-effective memory cache. In *18th USENIX Conference on File and Storage Technologies (FAST 20)*, pages 267–281, Santa Clara, CA, Feb. 2020. USENIX Association.

[46] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift. Peeking behind the curtains of serverless platforms. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 133–146, Boston, MA, July 2018. USENIX Association.

[47] N. J. Yadwadkar, B. Hariharan, J. E. Gonzalez, B. Smith, and R. H. Katz. Selecting the best vm across multiple public clouds: A data-driven performance modeling approach. In *Proceedings of the 2017 Symposium on Cloud Computing*, SoCC '17, pages 452–465, New York, NY, USA, 2017. ACM.

[48] M. Yan, P. Castro, P. Cheng, and V. Ishakian. Building a chatbot with serverless computing. In *Proceedings of the 1st International Workshop on Mashups of Things and APIs*, MOTA '16, New York, NY, USA, 2016. Association for Computing Machinery.

[49] Y. Yang, G.-W. Kim, W. W. Song, Y. Lee, A. Chung, Z. Qian, B. Cho, and B.-G. Chun. Pado: A data processing engine for harnessing transient resources in datacenters. In *Proceedings of the Twelfth European Conference on Computer Systems*, EuroSys '17, page 575–588, New York, NY, USA, 2017. Association for Computing Machinery.

[50] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation*, NSDI'12, page 2, USA, 2012. USENIX Association.